

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

04

제어문 이해하기



04-1 if 문

04-2 switch/case 문

04-3 while 문

04-4 for 문

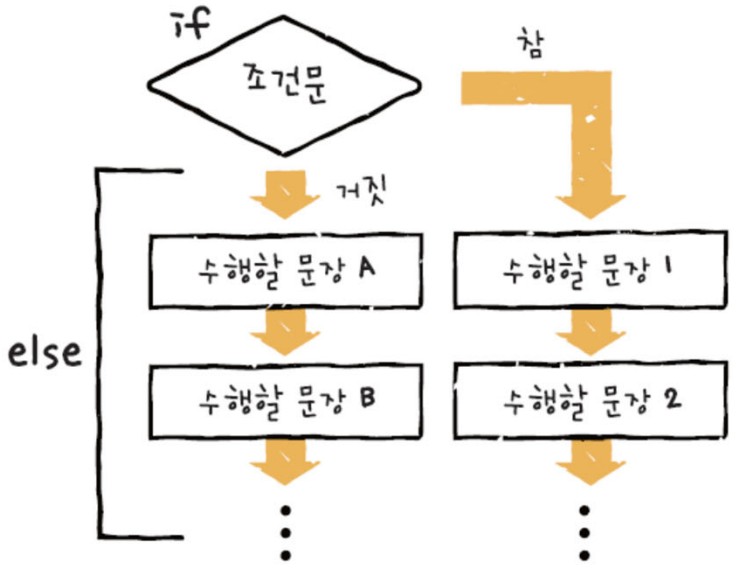
04-5 for each 문

04-1 if 문

- 조건을 판단하여 해당 조건에 맞는 상황을 수행하는 데 쓰이는 것

- **if 문과 else 문의 기본 구조**

```
if (조건문) {
    <수행할 문장1>;
    <수행할 문장2>;
    ...
} else {
    <수행할 문장A>;
    <수행할 문장B>;
    ...
}
```

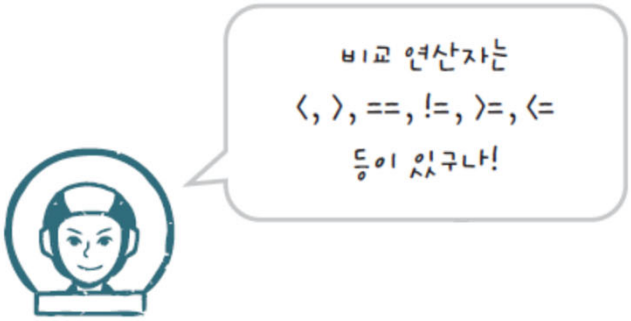


04-1 if 문

▪ 비교 연산자

▪ 조건 • 판단에 사용되는 비교 연산자

비교 연산자	설명
x < y	x가 y보다 작다
x > y	x가 y보다 크다
x == y	x와 y가 같다
x != y	x와 y가 같지 않다
x >= y	x가 y보다 크거나 같다
x <= y	x가 y보다 작거나 같다



04-1 if 문

▪ 비교 연산자

▪ 택시 예제 코딩하기

"만약 3000원 이상의 돈을 가지고 있으면 택시를 타고, 그렇지 않으면 걸어가라."

Do it! 코딩해 보세요

```
int money = 2000;
if (money >= 3000) {
    System.out.println("택시를 타고 가라");
}else {
    System.out.println("걸어가라");
}
```

실행 결과

```
걸어가라
```

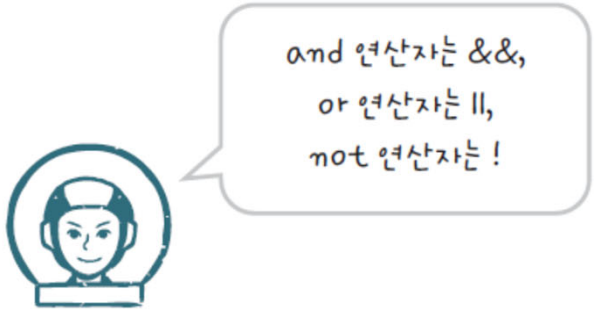
- money >= 3000 이란 조건문이 거짓이 되기 때문에 else 문의 문장이 수행됨

04-1 if 문

▪ and, or, not 연산자

- 조건 • 판단에 사용되는 and, or, not 연산자

연산자	설명
x && y	x와 y 모두 참이어야 참이다
x || y	x와 y 둘 중 적어도 하나가 참이면 참이다
!x	x가 거짓이면 참이다



04-1 if 문

and, or, not 연산자

택시 예제 코딩하기

"돈이 3000원 이상 있거나 카드가 있다면 택시를 타고, 그렇지 않으면 걸어가라."

Do it! 코딩해 보세요

```
int money = 2000;
boolean hasCard = true;

if (money>=3000 || hasCard) {
    System.out.println("택시를 타고 가라");
}else {
    System.out.println("걸어가라");
}
```

실행 결과

```
택시를 타고 가라
```

- money는 2000으로 3000보다 작지만 hasCard가 true이기 때문에 money >= 3000 || hasCard 조건문이 참이 됨

04-1 if 문

- **contains**
  - List 자료형에 해당 아이템이 있는지 조사하는 메서드
  - 택시 예제 코딩하기

> “만약 주머니에 돈이 있으면 택시를 타고, 없으면 걸어가라.”

04-1 if 문

- **contains**
  - 택시 예제 코딩하기
    - pocket 리스트에 안에 money란 요소가 있으므로 pocket.contains("money")가 참이 됨

Do it! 코딩해 보세요

```
import java.util.ArrayList;

public class Sample {
    public static void main(String[] args) {
        ArrayList<String> pocket = new ArrayList<String>();
        pocket.add("paper");
        pocket.add("cellphone");
        pocket.add("money");

        if (pocket.contains("money")) {
            System.out.println("택시를 타고 가라");
        }else {
            System.out.println("걸어가라");
        }
    }
}
```

실행 결과

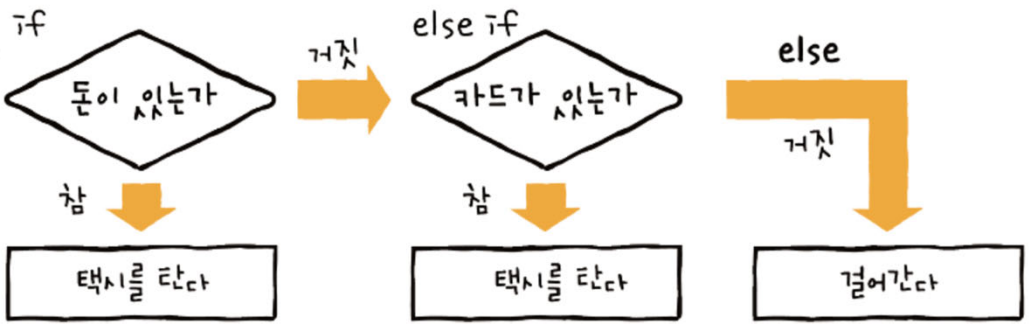
```
택시를 타고 가라
```

04-1 if 문

- **else if**
  - 이전 조건문이 거짓일 때 수행
  - 여러 조건을 판단할 수 있음

  - 택시 예제 코딩하기

> "지갑에 돈이 있으면 택시를 타고, 지갑엔 돈이 없지만 카드가 있으면 택시를 타고,
> 돈도 없고 카드도 없으면 걸어가라."



04-1 if 문

- **else if**
  - 택시 예제 코딩하기
    - pocket.contains("money") 조건은 거짓이므로 else if 문이 수행되고, hasCard 조건은 true가 됨

Do it! 코딩해 보세요

```
(... 생략 ...)
boolean hasCard = true;
ArrayList<String> pocket = new ArrayList<String>();
pocket.add("paper");
pocket.add("cellphone");

if (pocket.contains("money")) {
   System.out.println("택시를 타고 가라");
}else if(hasCard) {
   System.out.println("택시를 타고 가라");
}else {
   System.out.println("걸어가라");
}
```

실행 결과

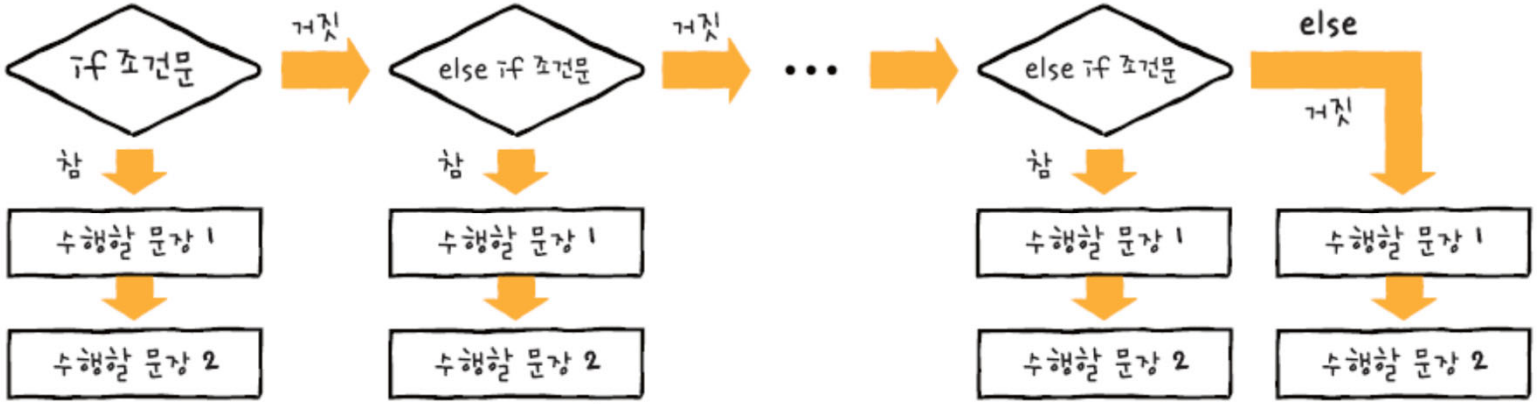
```
택시를 타고 가라
```

04-1 if 문

▪ if, else if, else의 기본 구조

```
if (조건문) {
    <수행할 문장1>
    <수행할 문장2>
    ...
} else if (조건문) {
    <수행할 문장1>
    <수행할 문장2>
    ...
} else if (조건문) {
    <수행할 문장1>
    <수행할 문장2>
    ...

} else {
    <수행할 문장1>
    <수행할 문장2>
    ...
}
```



04-2 switch/case 문

- if 문과 비슷하지만 좀 더 일정한 형식이 있는 조건·판단문
  - 입력 변수의 값과 일치하는 case 입력값 (입력값1, 입력값2, ...)이 있다면 해당 case 문에 속한 문장들이 실행
  - case 문마다 있는 break는 해당 case 문을 실행한 뒤 switch 문을 빠져나가기 위한 것
  - 입력값이 정형화되어 있는 경우 if 문보다는 switch/case 문을 쓰는 것이 코드 가독성이 좋음

```
switch(입력 변수) {
    case 입력값1: ...
        break;
    case 입력값2: ...
        break;
    ...
    default: ...
        break;
}
```

04-2 switch/case 문

Do it! 코딩해 보세요

```
public class Sample {
    public static void main(String[] args) {
        int month = 8;
        String monthString = "";
        switch (month) {
            case 1:  monthString = "January";
                break;
            case 2:  monthString = "February";
                break;
            case 3:  monthString = "March";
                break;
            case 4:  monthString = "April";
                break;
            case 5:  monthString = "May";
                break;
            case 6:  monthString = "June";
                break;
            case 7:  monthString = "July";
                break;
            case 8:  monthString = "August";
                break;
            case 9:  monthString = "September";
                break;
            case 10: monthString = "October";
                break;
            case 11: monthString = "November";
                break;
            case 12: monthString = "December";
                break;
            default: monthString = "Invalid month";
                break;
        }
        System.out.println(monthString);
    }
}
```

입력 변수의 자료형은 byte, short, char, int, enum, String만 가능하다.

실행 결과

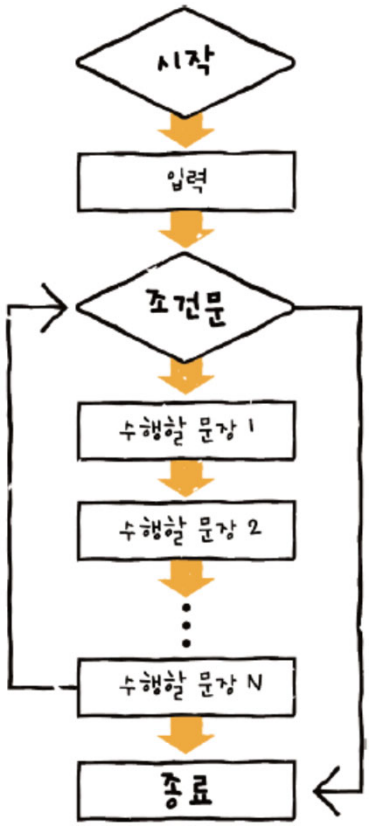
```
August
```

04-3 while 문

▪ while 문의 기본 구조

- 조건문이 참인 동안 while 문이 수행할 문장들을 반복하여 수행

```
while (조건문) {
    <수행할 문장1>;
    <수행할 문장2>;
    <수행할 문장3>;
    ...
}
```



04-3 while 문

while 문의 기본 구조

- '열 번 찍어 안 넘어가는 나무 없다'는 속담을 while 문을 활용해 코딩하기

Do it! 코딩해 보세요

```
int treeHit = 0;
while (treeHit < 10) {
    treeHit++;
    System.out.println("나무를  " + treeHit + "번 찍었습니다.");
    if (treeHit == 10) {
        System.out.println("나무 넘어갑니다.");
    }
}
```

treeHit += 1;로도 표현 가능

실행 결과

```
나무를  1번 찍었습니다.
나무를  2번 찍었습니다.
나무를  3번 찍었습니다.
나무를  4번 찍었습니다.
나무를  5번 찍었습니다.
나무를  6번 찍었습니다.
나무를  7번 찍었습니다.
나무를  8번 찍었습니다.
나무를  9번 찍었습니다.
나무를  10번 찍었습니다.
나무 넘어갑니다.
```

04-3 while 문

- **무한루프(loop)란?**
  - 무한히 반복한다는 의미
  - 무한 루프의 개념을 사용하지 않는 프로그램은 하나도 없을 정도로 자주 사용됨

```
while (true) {
    <수행할 문장1>;
    <수행할 문장2>;
    ...
}
```

  - 조건문 자체가 true이므로 조건문은 항상 참

04-3 while 문

무한루프(loop)란?

- 무한히 반복하는 코딩 예

Do it! 코딩해 보세요

```
while (true) {
    System.out.println("Ctrl-C를 눌러야 while문을 빠져나갈 수 있습니다.");
}
```

실행 결과

```
Ctrl-C를 눌러야 while문을 빠져나갈 수 있습니다.
Ctrl-C를 눌러야 while문을 빠져나갈 수 있습니다.
(... 생략 ...)
```

04-3 while 문

while 문 빠져나가기 – break

- while 문을 강제로 멈춰야 할 때 사용

Do it! 코딩해 보세요

```
int coffee = 10;
int money = 300;

while (money > 0) {
    System.out.println("돈을 받았으니 커피를 줍니다.");
    coffee--;
    System.out.println("남은 커피의 양은 " + coffee + "입니다.");
    if (coffee == 0) {
        System.out.println("커피가 다 떨어졌습니다. 판매를 중지합니다.");
        break;
    }
}
```

실행 결과

```
돈을 받았으니 커피를 줍니다.
남은 커피의 양은 9입니다.
돈을 받았으니 커피를 줍니다.

(... 생략 ...)

남은 커피의 양은 1입니다.
돈을 받았으니 커피를 줍니다.
남은 커피의 양은 0입니다.
커피가 다 떨어졌습니다. 판매를 중지합니다.
```

04-3 while 문

while 문으로 돌아가기 – continue

- while 문 안의 문장을 수행할 때 어떤 조건을 검사해서 참이 아닌 경우 while 문을 빠져나가는 대신 while 문의 맨 처음, 즉 조건문으로 돌아가게 하고 싶은 경우에 사용

Do it! 코딩해 보세요

```
int a = 0;
while (a < 10) {
    a++;
    if (a % 2 == 0) {
        continue;  // 짝수인 경우 조건문으로 돌아간다.
    }
    System.out.println(a);  // 홀수만 출력된다.
}
```

실행 결과

```
1
3
5
7
9
```

04-4 for 문

for 문의 기본 구조

- while 문과 마찬가지로 문장을 반복해서 수행해야 할 경우 사용
- 세미콜론(;)을 구분자로 사용

```
for (초깃값; 조건문; 증갓값) {
    ...
}
```

Do it! 코딩해 보세요

```
String[] numbers = {"one", "two", "three"};
for(int i=0; i<numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

실행 결과

```
one
two
three
```

04-4 for 문

▪ for 문의 기본 구조

▪ for 문 예시

"총 5명의 학생이 시험을 보았는데 시험 점수가 60점이 넘으면 합격이고, 그렇지 않으면 불합격이다. 합격, 불합격을 판단하여 출력하시오."

Do it! 코딩해 보세요

```
int[] marks = {90, 25, 67, 45, 80};
for(int i=0; i<marks.length; i++) {
    if (marks[i] >= 60) {
        System.out.println((i+1)+"번 학생은 합격입니다.");
    }else {
        System.out.println((i+1)+"번 학생은 불합격입니다.");
    }
}
```

실행 결과

```
1번 학생은 합격입니다.
2번 학생은 불합격입니다.
3번 학생은 합격입니다.
4번 학생은 불합격입니다.
5번 학생은 합격입니다.
```

04-4 for 문

for 문으로 돌아가기 – continue

- for 문 안의 문장을 수행하는 도중에 continue 문을 만나면 for 문의 처음으로 돌아감

Do it! 코딩해 보세요

```
int[] marks = {90, 25, 67, 45, 80};
for(int i=0; i<marks.length; i++) {
    if (marks[i] < 60) {
        continue;  // 조건문으로 돌아간다.
    }
    System.out.println((i+1)+"번 학생 축하합니다. 합격입니다.");
}
```

실행 결과

```
1번 학생 축하합니다. 합격입니다.
3번 학생 축하합니다. 합격입니다.
5번 학생 축하합니다. 합격입니다.
```

04-4 for 문

▪ 이중 for 문

- for 문을 두 번 사용하여 간단하게 출력하는 구구단

Do it! 코딩해 보세요

```
for(int i=2; i<10; i++) {
    for(int j=1; j<10; j++) {
        System.out.print(i*j+" ");
    }
    System.out.println("");  // 줄을 바꾸어 출력하는 역할을 한다.
}
```

실행 결과

```
2 4 6 8 10 12 14 16 18
3 6 9 12 15 18 21 24 27
4 8 12 16 20 24 28 32 36
5 10 15 20 25 30 35 40 45
6 12 18 24 30 36 42 48 54
7 14 21 28 35 42 49 56 63
8 16 24 32 40 48 56 64 72
9 18 27 36 45 54 63 72 81
```

줄을 바꾸어 출력

- 1부터 9까지의 숫자가 j에 대입되고
  그 다음 문장인 System.out.print(i*j+" ");를 수행

04-5 for each 문

for each 문

- J2SE 5.0부터 추가됨
- for 문을 이용하지만 조건문의 문법이 조금 다름

```
for (type 변수명: iterate) {
    body-of-loop
}
```

  - iterate 객체에서 한 개씩 순차적으로 변수명에 대입되어 for 문 수행
  - iterate에는 루프를 돌릴 수 있는 자료형(배열이나 ArrayList 등)만 가능
  - 변수명의 type(자료형)은 iterate 객체에 포함된 자료형과 일치해야 함

04-5 for each 문

for each 문

ArrayList로 구현한 예제

Do it! 코딩해 보세요

```
import java.util.ArrayList;
import java.util.Arrays;

public class Sample {
    public static void main(String[] args) {
        ArrayList<String> numbers = new ArrayList<>(Arrays.asList("one", "two",
"three"));
        for (String number: numbers) {
            System.out.println(number);
        }
    }
}
```

실행 결과

```
one
two
three
```



감사합니다.